

Setup guide: Python, Positron, Quarto and GitHub

International Marketing Analytics · WU Vienna

We go through this guide together in Session 1; you do not need to install anything beforehand. Bring your laptop and charger, and know the password of your WU student e-mail account. The slides [slides/01a_setup-and-registration_WT26-27.pdf](#) show the same steps.

0. Register your accounts

1. **GitHub:** create a free account at <https://github.com/signup> with your WU student e-mail, pick a professional username (it appears on every commit) and turn on two-factor authentication.
2. **GitHub Education:** apply for student benefits at <https://education.github.com/pack> (verified WU e-mail plus proof of enrolment). Approval can take a few days; the free Copilot Student plan becomes available once approved. Until then, Copilot Free works.
3. **DataCamp:** join the course classroom through the invitation link on Canvas, using your WU e-mail. This gives you free access to the assigned courses.

1. Install Python 3.14

The course uses the latest Python, **3.14.8** (30 September 2026). Windows and macOS do not come with Python, so you install it once. Both downloads are on <https://www.python.org/downloads/>.

- **Windows:** download the **Python install manager** and double-click it to install. Then, in a terminal, run `py install 3.14`. The install manager installs Python itself and keeps it up to date.
- **macOS:** download the **macOS installer** for Python 3.14.8 (Apple silicon and Intel), open the `.pkg` and click through.

Check in a new terminal:

```
py -3.14 --version      # Windows
python3.14 --version    # macOS
```

Alternatively, let Positron install Python for you once Positron is installed: *Command Palette* → *Python: Install Python via uv*.

2. Install Positron

Positron is a data-science editor from Posit with first-class Python and Quarto support. Download it from <https://positron.posit.co/download> (current version 2026.09) and install it. Its welcome page checks whether Python is ready.

3. Install Quarto

Download and install Quarto from <https://quarto.org/docs/get-started/>. Restart Positron afterwards so it finds Quarto.

4. Install GitHub Desktop

Install GitHub Desktop from <https://desktop.github.com> and sign in with your GitHub account. It brings its own Git and gives you a point-and-click interface to clone and pull the course repository (commit and push if your group shares a repository).

5. Clone the course repository

In GitHub Desktop: *File* → *Clone repository* → *URL* and paste the repository address from Canvas. Choose a folder without spaces, umlauts or cloud syncing (OneDrive, iCloud), for example `C:\Users\you\wu\ima` or `~/wu/ima`. Then open that folder in Positron with *File* → *Open Folder*.

6. Set up the project's Python

The cloned course folder is your **project** in Positron. Give it its own Python environment:

1. *Command Palette* (Ctrl/Cmd + Shift + P) → **Python: Create Environment**
2. Choose **Venv**, then **Python 3.14.8**
3. Tick **requirements.txt** so Positron installs all course packages
4. Positron creates `.venv` inside the project and selects it in the **Interpreter picker** (top right): *Python 3.14.8 (.venv)*

This installs everything the course uses, including `polars` (data wrangling), `plotnine` (charts), `great_tables` (tables), `statsmodels` (regression) and `pymc-marketing` (Bayesian MMM). The versions are pinned in `requirements.txt`, so everyone gets the same results. If the packages were not installed, run this in the Positron console (Windows and macOS alike):

```
%pip install -r requirements.txt
```

For a new project of your own, use *New* ▼ → *New Folder from Template* → *Python Project*. Positron creates the folder, a `.venv` and Git in one step.

Install and load single packages

Three ways to install, all into the course `.venv`:

```
%pip install polars                                # Positron console, both systems
.venv\Scripts\python -m pip install polars          # Windows terminal
.venv/bin/python -m pip install polars              # macOS terminal
```

Or use the **Packages** pane (package icon in the Activity Bar → *Install Package*). The install name can differ from the import name: `great-tables` → `great_tables`, `scikit-learn` → `sklearn`.

Loading is the same on both systems, at the top of every file:

```
import polars as pl
import statsmodels.formula.api as smf
from plotnine import ggplot, aes, geom_line
from great_tables import GT
```

7. Connect your AI assistant

Positron's built-in AI is **Posit Assistant** (since Positron 2026.07; it replaced Positron Assistant).

1. Command Palette → *Authentication: Configure Language Model Providers* → **GitHub Copilot** → sign in with GitHub. Copilot gives chat and code completions; it is free with Copilot Student (after GitHub Education approval) or Copilot Free.
2. Open the chat with *View: Show Posit Assistant*.

Optional: Claude inside Positron

The course default is GitHub Copilot, free with GitHub Education. If you want to use Claude instead, there are two ways; both cost money.

- **Claude as Posit Assistant's model:** *Command Palette* → *Authentication: Configure Language Model Providers* → *Anthropic*, then paste an API key from the Claude Console (*API keys* → *Create Key*). API use is paid per use; a Claude Pro or Max subscription does not include API access. Pick a Claude model in the chat.
- **Claude Code, Anthropic's coding agent:** install the *Claude Code* extension (publisher Anthropic) from the Extensions view, or install the command-line tool in Positron's terminal (Windows PowerShell: `irm https://claude.ai/install.ps1 | iex`; macOS: `curl -fsSL https://claude.ai/install.sh | bash`) and start it with `claude` in your project folder. Sign in with a paid Claude plan or a Claude Console account. Claude Code reads your project, runs code and asks before it changes a file; a `CLAUDE.md` file in the project holds standing instructions such as “use polars, plotnine, great_tables and statsmodels only”.

Never share an API key, and never paste personal data into any assistant.

The rule for this course: the AI writes, you check. Run the code, read the output and export the chat as your prompt log.

8. The GitHub workflow you will use

1. **Pull** before every session (GitHub Desktop: *Fetch origin*, then *Pull origin*). This brings the latest course materials.
2. **Do not edit inside the course repository.** Copy the files you want to change into your own folder; otherwise your edits clash with our updates when you pull.
3. **Optional: a group repository.** If your group wants to share work through GitHub, one member publishes a project folder as a private repository (*Repository* → *Publish repository*) and invites the others (on github.com: *Settings* → *Collaborators*). Then: pull before you start, commit small steps with a message that says what changed (“add TV adstock to German model”), push when you stop, one file per person where possible. If GitHub Desktop reports a conflict, ask your AI assistant to explain the conflict markers, keep the version you want, commit, push.
4. Sharing files another way is fine too: only what you upload to Canvas is assessed, not your commit history.

9. Check your installation

Run in the Positron console:

```
%run setup/check_setup.py
```

or open `setup/check_setup.py` and press *Run*. The script lists Python, each course package, Quarto and Git, and ends with `ready`. A line marked `!!` names what is still missing.

Then test the whole stack with `test_stack.qmd`, a single file that also works outside the course folder (it reads the data from GitHub, so you need internet). Open it in Positron, run its first cell once with Ctrl/Cmd + Enter (it installs polars, plotnine, great_tables, statsmodels, pyarrow and jupyter), then press *Preview* (Ctrl/Cmd + Shift + K). A table, a chart and regression results appear: you are ready for Session 1.

10. Update within Positron

- **Positron** updates itself: on Windows and macOS it downloads new versions in the background and applies them when you restart. To check by hand: *Help* → *Check for Updates*.
- **Python:** *Command Palette* → *Python: Install Python via uv* installs any supported version (up to 3.14) next to the ones you have.
- **Packages:** after pulling a new `requirements.txt`, run `%pip install -r requirements.txt` in the console; a single package with `%pip install --upgrade polars`.
- **Course materials** do not come through Positron: pull them in GitHub Desktop before every session.

Move a project to Python 3.14

A `.venv` belongs to the Python that created it and cannot be upgraded; you rebuild it. If the check shows Python 3.13 or older:

1. *Command Palette* → *Python: Install Python via uv* → 3.14.

2. Stop the Python session (power icon in the Console).
3. In the Explorer, right-click the `.venv` folder → *Delete*. If the folder is in OneDrive or Dropbox, pause syncing first.
4. *Command Palette* → *Python: Create Environment* → Venv → Python 3.14.8, tick `requirements.txt`.
5. Pick *Python 3.14.8 (.venv)* in the Interpreter picker and run `%run setup/check_setup.py`: it reports 3.14.8 and ends with `ready`.

Terminal alternative on Windows: `py install 3.14`, then `Remove-Item -Recurse -Force .venv` before step 4.

Troubleshooting

- **The check shows Python 3.13 or older:** move the project to Python 3.14 (section 10).
- **Windows: `python` or `py` is not recognised:** reinstall the Python install manager, then run `py install 3.14`.
- **No `.venv` in the Interpreter picker:** open the course folder (not a single file), then pick *Python 3.14.8 (.venv)*.
- **macOS: `externally-managed-environment`:** you used the system Python; select the project's `.venv` first.
- **PyMC compiles slowly on first run:** this is normal (a C compiler is used once).
- **Quarto cannot find Python:** check that the `.venv` interpreter is selected; in the terminal run `quarto check jupyter`.